

PDF 2 - Product, UX & Frontend Blueprint

A polished learner journey with excellent UI, minimal re-renders, clear states and accessible interactions

CipherSchools Hiring Assignment - LLD Practice Platform | September 2026

1. UX principle

Design target

The interface should feel like a focused coding-practice product, not an LMS. Every screen must move the learner through the loop: choose -> think -> submit -> understand feedback -> improve.

2. Screens

Route	Purpose	Key UI
/	Problem library	Search/filter, difficulty, tags, last attempt, best score
/problems/[slug]	Read and start	Requirements, constraints, evaluation rubric preview, Start/Continue
/attempts/[id]	Practice workspace	Problem left rail; structured design editor; autosave status; submit CTA
/attempts/[id]/feedback	Explainable evaluation	Overall summary, rubric cards, evidence, suggestions, retry button
/history	Learning history	Attempts, status, score trend, recurring weakness chips
/compare?a=&b=	Compare attempts	Side-by-side criterion delta and changed design sections

3. Practice workspace layout

- Desktop: 35% problem context / 65% design editor with sticky top action bar.
- Editor sections: Assumptions; Core classes & responsibilities; Relationships/interfaces; Main flow; Edge cases; Trade-offs/extensibility.
- Autosave indicator: Saved / Saving / Offline changes.
- Submission modal shows completeness checks before creating immutable snapshot.
- Mobile/tablet collapses problem context into a drawer; editor remains single-column.

4. Component architecture

app/

(app)/problems/page.tsx

(app)/problems/[slug]/page.tsx

(app)/attempts/[id]/page.tsx
 (app)/attempts/[id]/feedback/page.tsx
 (app)/history/page.tsx
 components/
 problem/ProblemCard.tsx ProblemRequirements.tsx
 practice/PracticeShell.tsx DesignEditor.tsx EditorSection.tsx AutosaveStatus.tsx
 feedback/FeedbackSummary.tsx RubricCard.tsx EvidenceQuote.tsx AttemptDelta.tsx
 history/AttemptTable.tsx ScoreTrend.tsx
 ui/ (shadcn primitives only)
 lib/
 queries/ mutations/ schemas/ domain/

5. Preventing unnecessary re-renders

Risk	Technique
Typing re-renders whole page	Keep editor state local; split each section into memoized child; do not lift keystrokes to page root
Autosave on every key	Debounce 700-1000ms; persist only changed section; cancel stale request
History fetch reruns	Server Components for initial data; TanStack Query only where client refresh is required
Large feedback tree	Render static feedback server-side; client components only for expand/collapse/compare interactions
Derived values recompute	Compute server-side or use memoization only for truly expensive derivations
Unstable props	Pass primitives/IDs; avoid inline object/function props to memoized children
Global store churn	No global form store; use route/server state for attempts and local state for active draft

6. Data-fetch strategy

AI feedback requests never run from client components. The browser submits to the application backend; the backend invokes Gemini with the server-only API key and persists the validated result. This prevents exposing the Google AI Studio key in the frontend bundle.

- Use Server Components for problem list/detail, history and completed feedback.
- Use Server Actions or route handlers for start attempt, autosave, submit and retry evaluation.
- Use optimistic UI only for safe actions such as section save indicator; never fake submission completion.
- After submit, poll evaluation status every 2-3 seconds with stop conditions Completed/Failed, or use server-sent updates only if already comfortable.

7. Visual system

Element	Recommendation
Palette	Deep navy / blue primary, neutral slate surfaces, green success, amber warning, red failure
Typography	Inter/Geist; 15-16px body; strong 24-32px page headings
Cards	Subtle borders, 12-16px radius, minimal shadow
Feedback	Criterion card: score ring + evidence + concern + concrete

	next action
Status	Submitted=neutral, Evaluating=blue animated dot, Completed=green, Failed=red with retry
Density	Information-rich but calm; avoid giant hero sections inside app

8. Accessibility and UX polish

- Keyboard-accessible editor sections and dialogs; visible focus rings.
- ARIA live region for autosave and evaluation status changes.
- Never rely on color alone for status.
- Preserve draft on navigation; warn only when unsaved local changes exist.
- Skeletons for first load; no layout shift for score cards.
- Empty states explain the next action instead of showing blank tables.

9. UI acceptance checklist

- A user understands the task within 10 seconds.
- The primary CTA is obvious on every step.
- No full-page rerender while typing.
- Submit cannot be double-clicked into duplicate processing.
- Feedback always cites evidence from the learner response.
- History makes retry/improvement the natural next step.